Experiment No:11

Using the Python Matplotlib library for data visualization, create a variety of charts.

In [ ]:
```
# importing packages
import numpy as np
import matplotlib.pyplot as plt

# create data
x = np.linspace(1, 50, 50)
np.random.seed(1)
y = np.random.randint(0, 20, 50)

# plot graph
plt.plot(x, y)

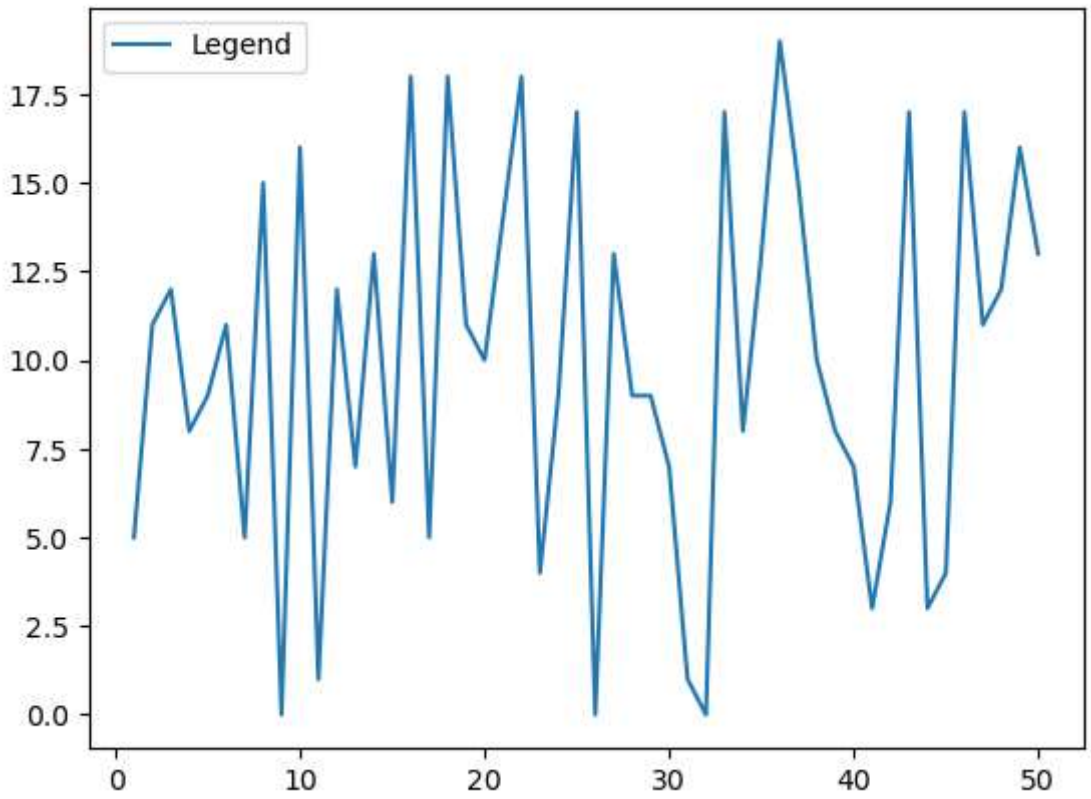
# add legend
plt.legend(['Legend'])
plt.show()
```

In [2]:
```
# importing packages
import numpy as np
import matplotlib.pyplot as plt

# create data
x = np.linspace(1, 50, 50)
np.random.seed(1)
y = np.random.randint(0, 20, 50)

# plot graph
plt.plot(x, y)

# add legend and set position to upper left
plt.legend(['Legend'], loc='upper left')
plt.show()
```



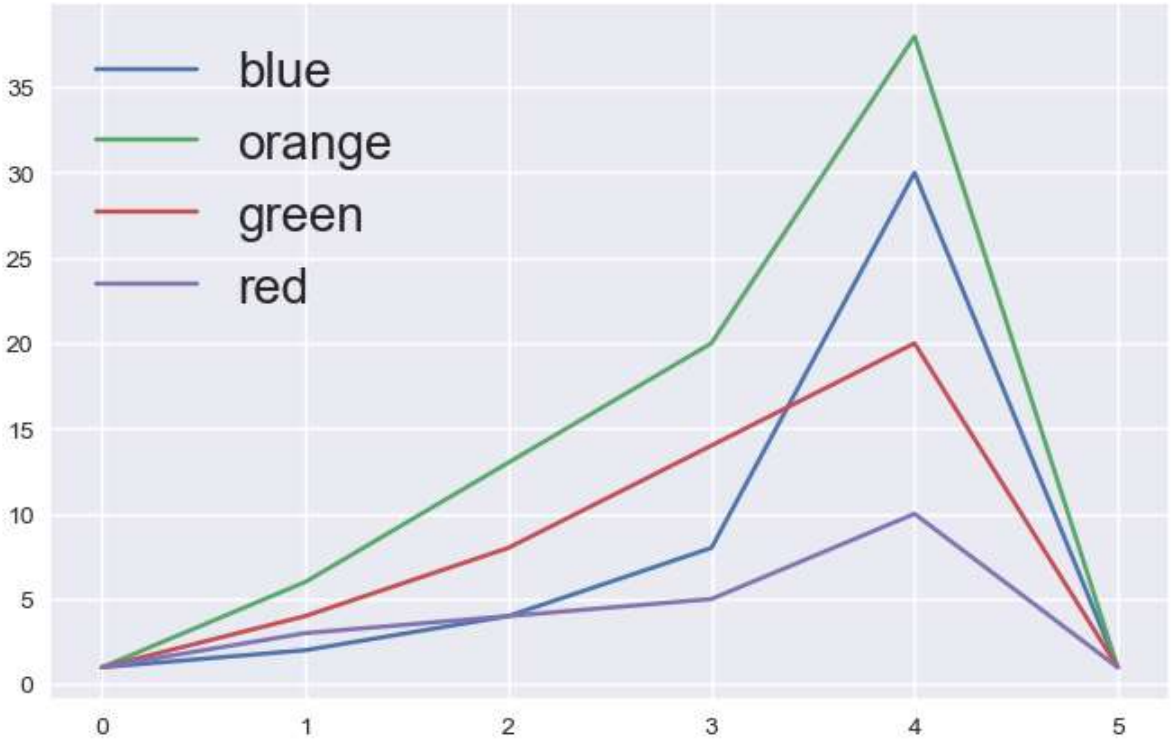
In [13]:
```
import matplotlib.pyplot as plt

plt.figure(figsize = (8 , 5))

plt.plot([1, 2, 4, 8, 30, 1])
plt.plot([1, 6, 13, 20, 38, 1])
plt.plot([1, 4, 8, 14, 20, 1])
plt.plot([1, 3, 4, 5, 10, 1])

#Using prop keyword in legend to change font size
plt.legend(['blue', 'orange', 'green', 'red'],
           prop = {'size' : 20},
           loc = 'upper left', shadow = True,
           facecolor = 'yellow')

plt.show()
```



In [3]:
```
# importing packages
import numpy as np
import matplotlib.pyplot as plt

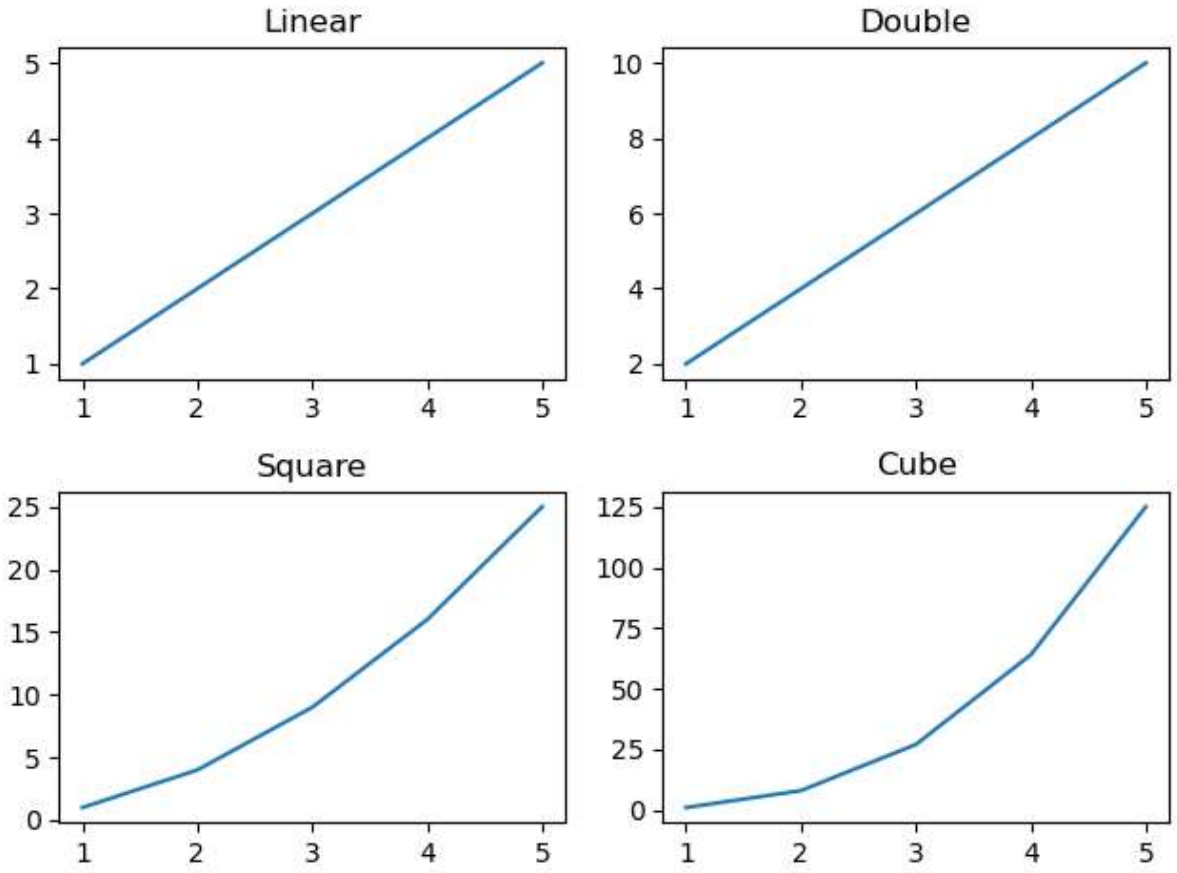
# create data
x=np.array([1, 2, 3, 4, 5])

# making subplots
fig, ax = plt.subplots(2, 2)

# set data with subplots and plot
ax[0, 0].plot(x, x)
ax[0, 1].plot(x, x*2)
ax[1, 0].plot(x, x*x)
ax[1, 1].plot(x, x*x*x)

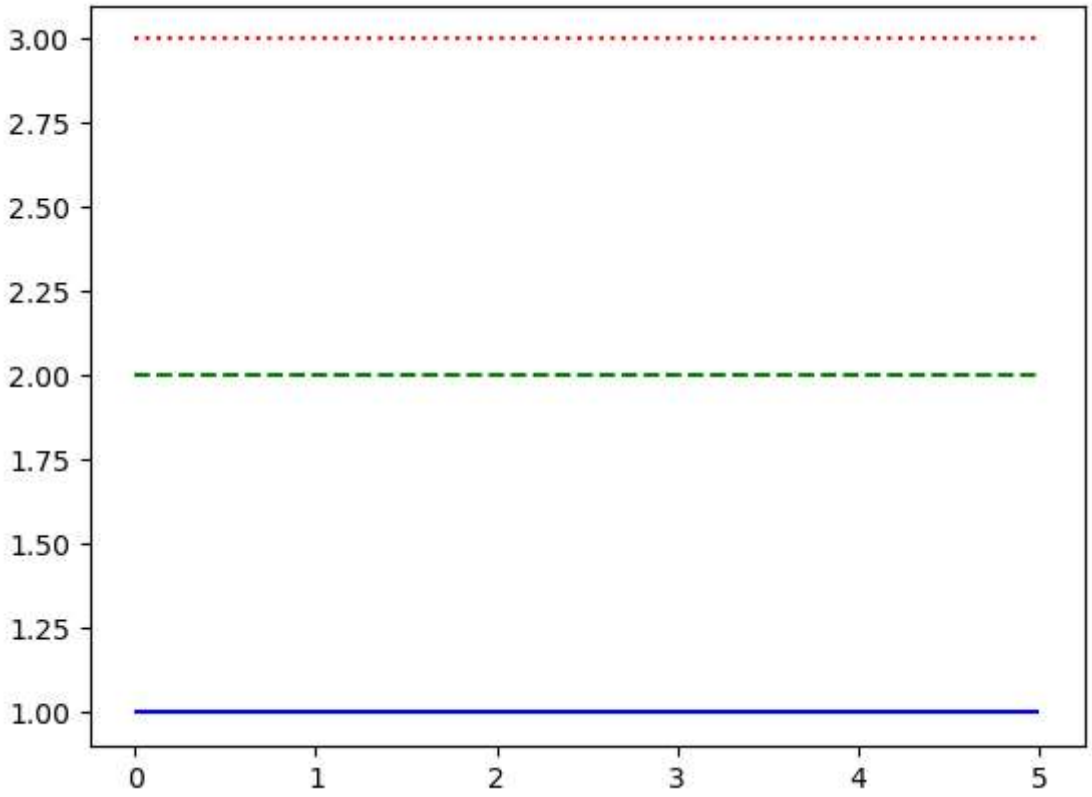
# set the title to subplots
ax[0, 0].set_title("Linear")
ax[0, 1].set_title("Double")
ax[1, 0].set_title("Square")
ax[1, 1].set_title("Cube")

# set spacing
fig.tight_layout()
plt.show()
```

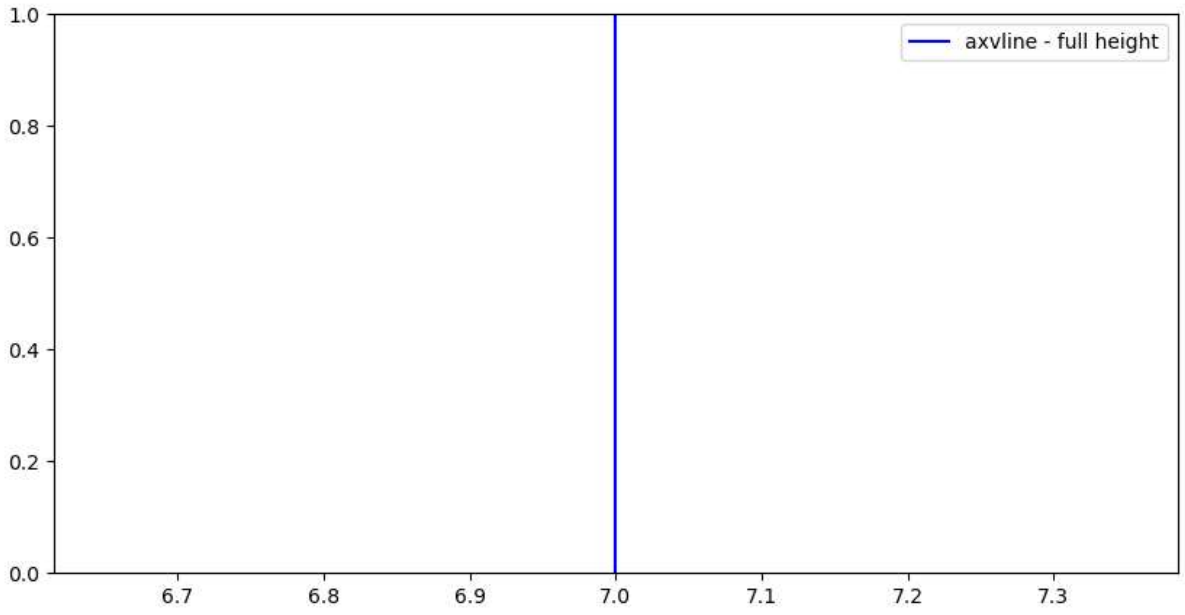


In [4]:
```
import matplotlib.pyplot as plt

# Draw horizontal lines at y=1, y=2 and y=3
plt.hlines(y=[1, 2, 3], xmin=0, xmax=5, colors=['b', 'g', 'r'], linestyles=['-
plt.show()
```



In [5]:
```
import matplotlib.pyplot as plt
plt.figure(figsize = (10, 5))
plt.axvline(x = 7, color = 'b', label = 'axvline - full height')
plt.legend()
plt.show()
```



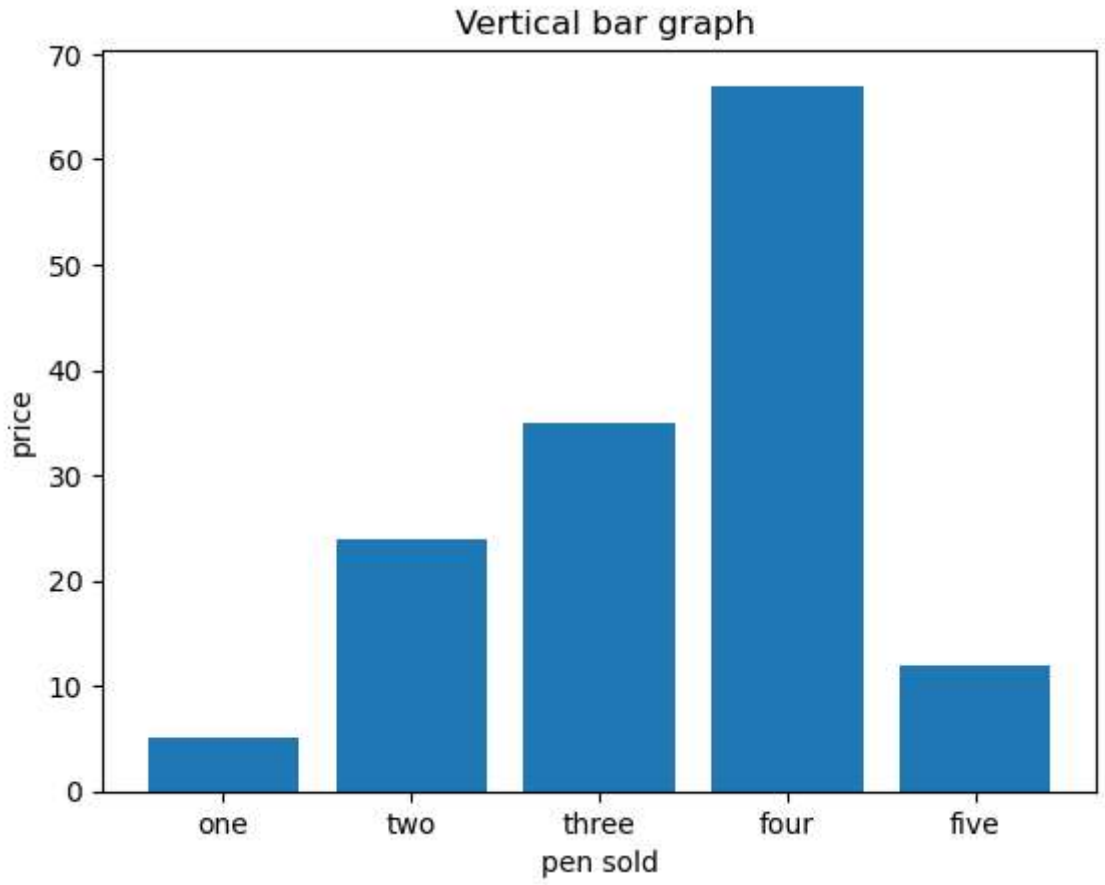
In [6]:
```
import matplotlib.pyplot as plt

x=['one', 'two', 'three', 'four', 'five']

# giving the values against
# each value at x axis
y=[5, 24, 35, 67, 12]
plt.bar(x, y)

# setting x-label as pen sold
plt.xlabel("pen sold")

# setting y_label as price
plt.ylabel("price")
plt.title(" Vertical bar graph")
plt.show()
```

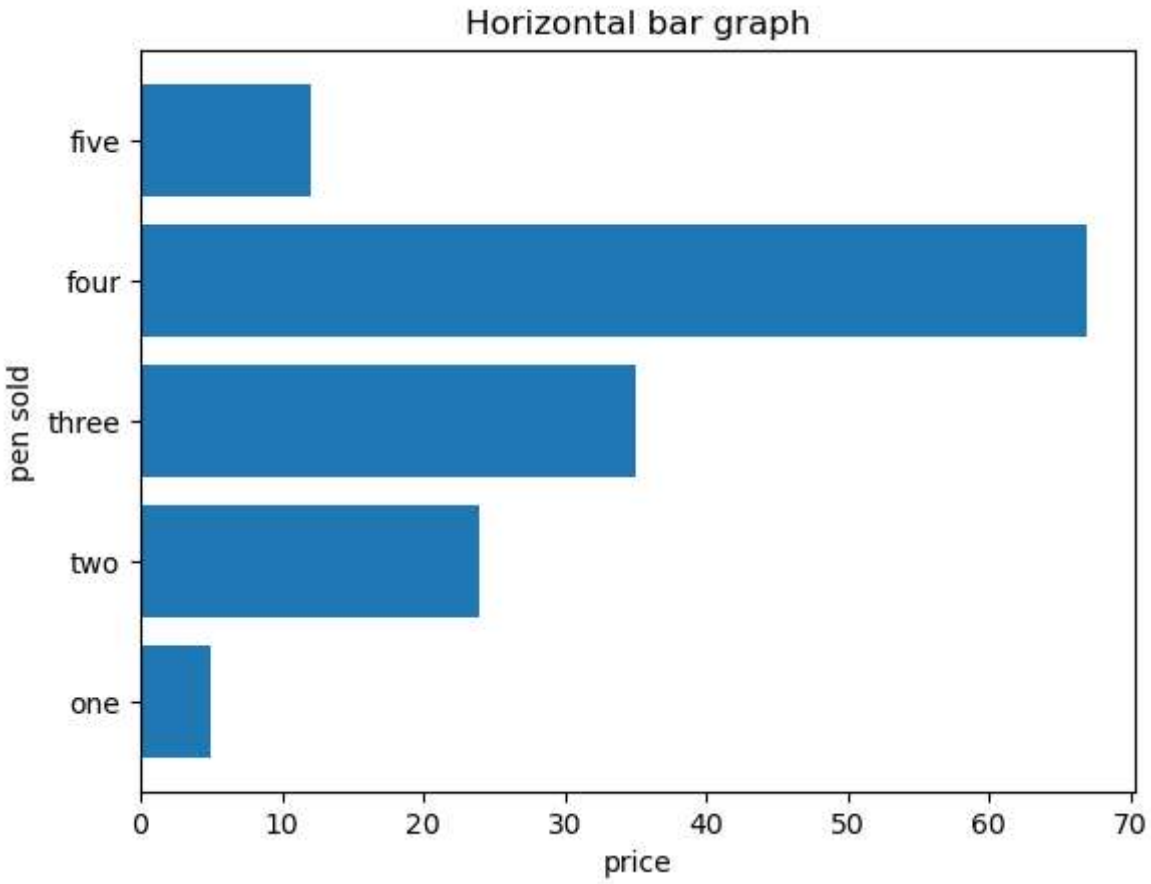


In [7]:
```
import matplotlib.pyplot as plt
y=['one', 'two', 'three', 'four', 'five']

# getting values against each value of y
x=[5,24,35,67,12]
plt.barh(y, x)

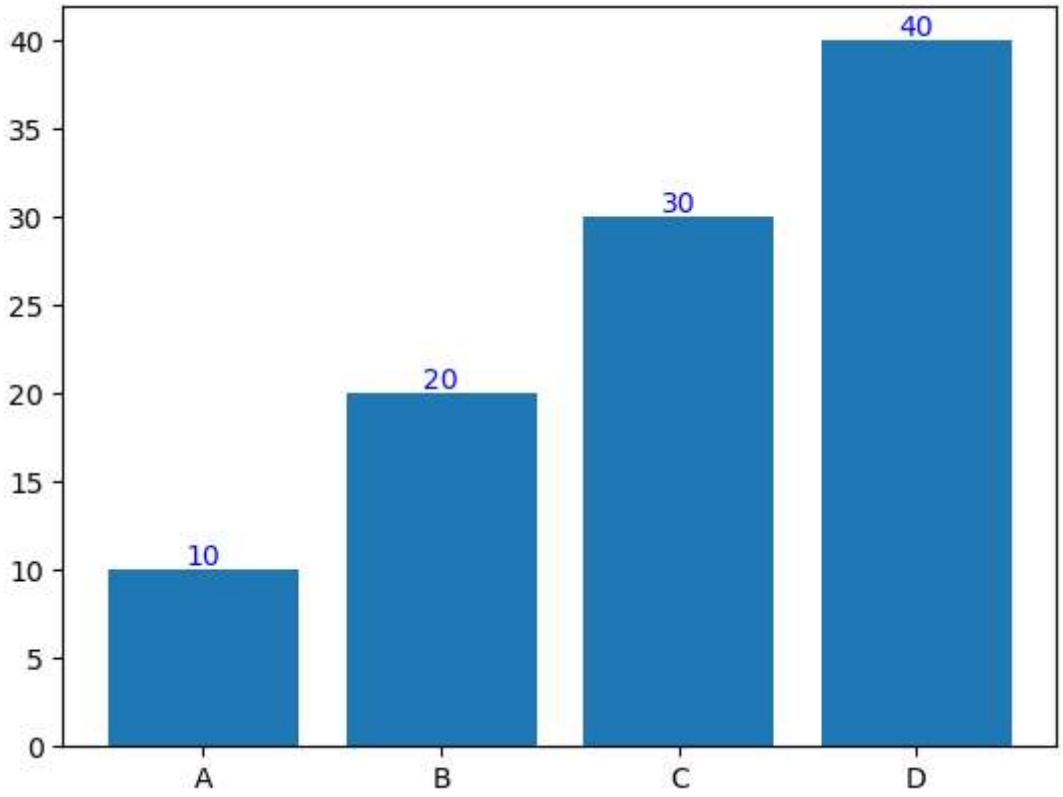
# setting label of y-axis
plt.ylabel("pen sold")

# setting label of x-axis
plt.xlabel("price")
plt.title("Horizontal bar graph")
plt.show()
```



In [8]:
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
bars = ax.bar(["A", "B", "C", "D"], [10, 20, 30, 40])

ax.bar_label(bars, label_type='edge', color='blue')
plt.show()
```

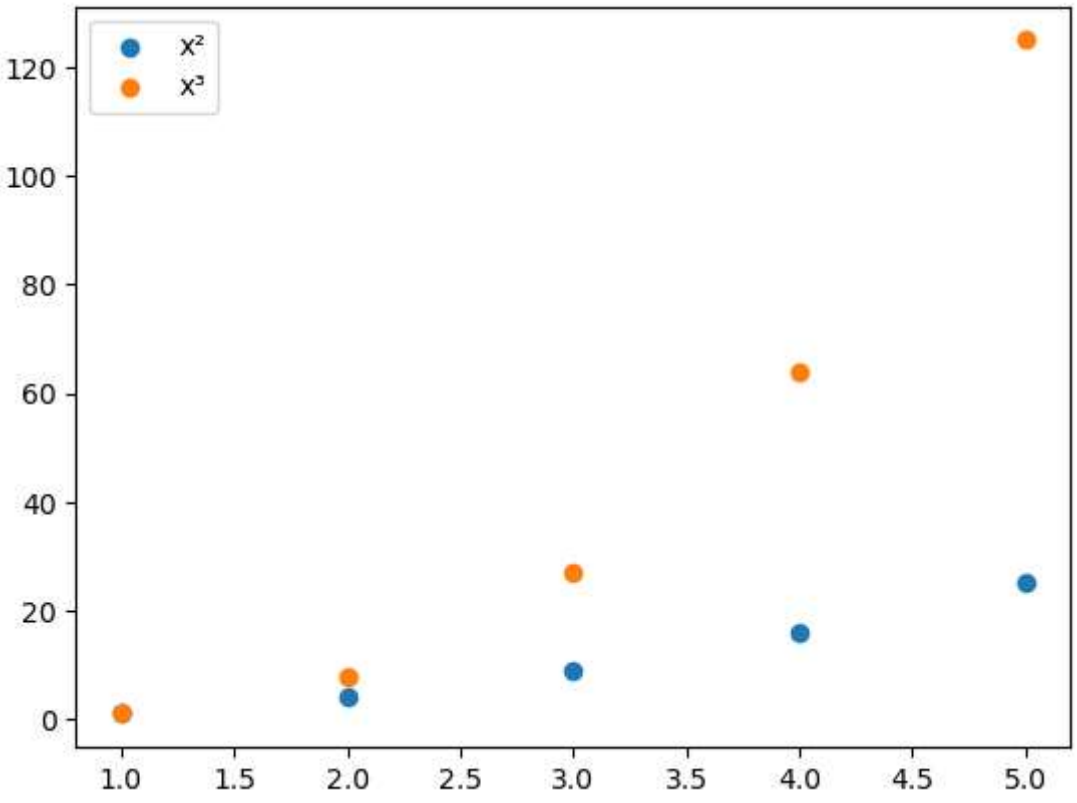


In [9]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(1, 6)
y1 = x**2
y2 = x**3

plt.scatter(x, y1, label="x²")
plt.scatter(x, y2, label="x³")

plt.legend()
plt.show()
```



In [10]:
```
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('seaborn')

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [8, 7, 6, 4, 5, 6, 7, 8, 9, 10]

plt.xticks(np.arange(11))
plt.yticks(np.arange(11))

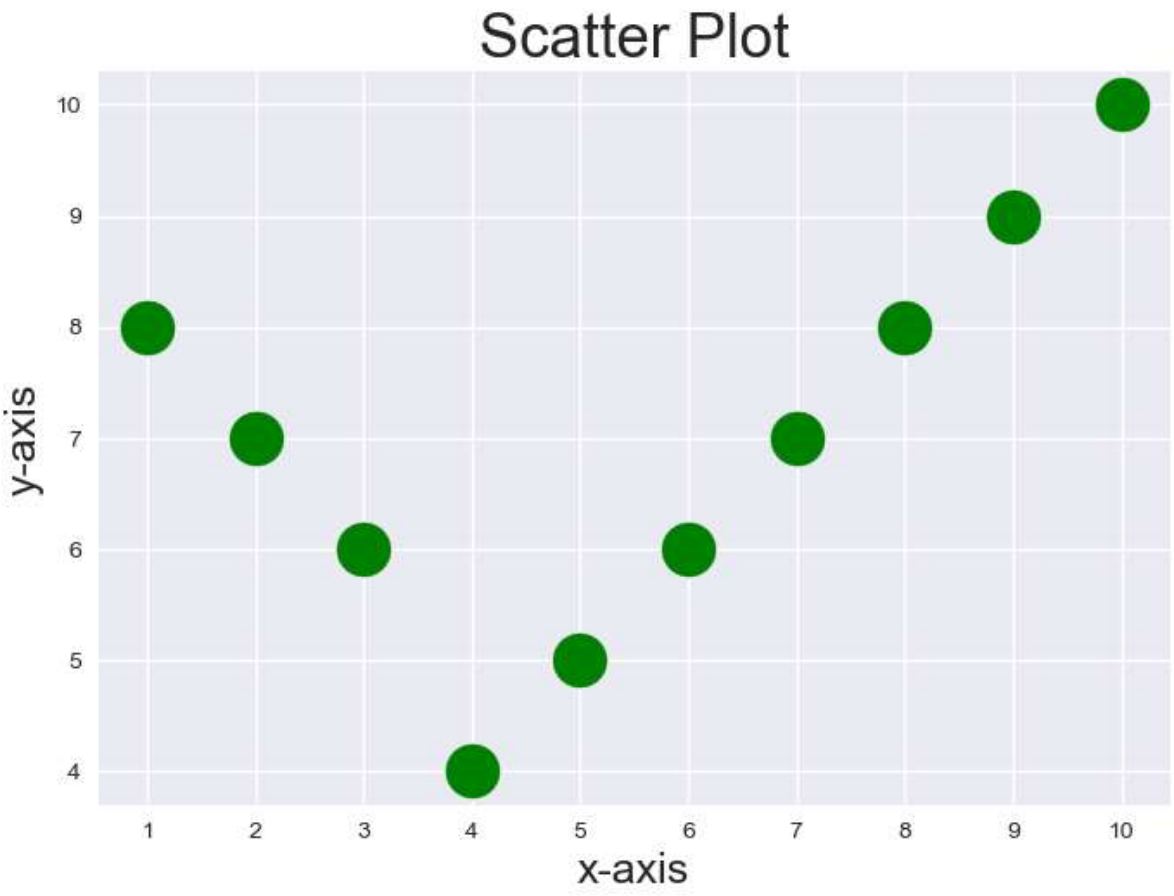
plt.scatter(x, y, s=500, c='g')

plt.title("Scatter Plot", fontsize=25)

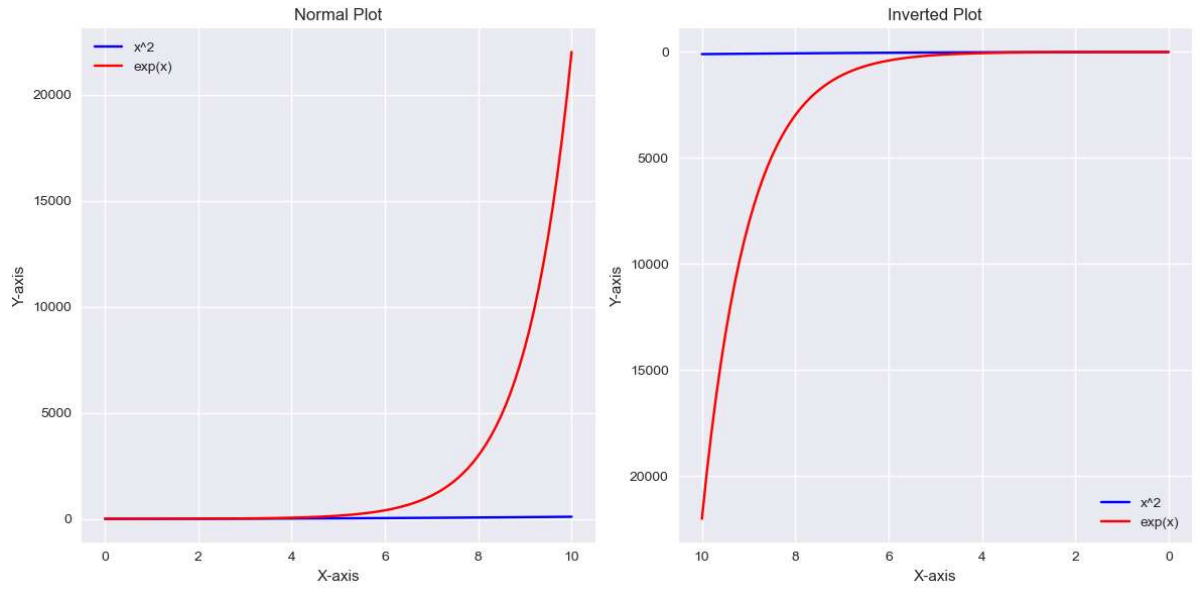
plt.xlabel('x-axis', fontsize=18)
plt.ylabel('y-axis', fontsize=18)

plt.show()
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_8028\3837555621.py:4: MatplotlibD
eprecationWarning: The seaborn styles shipped by Matplotlib are deprecated si
nce 3.6, as they no longer correspond to the styles shipped by seaborn. Howev
er, they will remain available as 'seaborn-v0_8-<style>'. Alternatively, dire
ctly use the seaborn API instead.
  plt.style.use('seaborn')
```



In [11]:
```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 10, 100)
y1 = x**2
y2 = np.exp(x)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
ax1.plot(x, y1, label="x^2", color='b')
ax1.plot(x, y2, label="exp(x)", color='r')
ax1.set_title("Normal Plot")
ax1.set_xlabel("X-axis")
ax1.set_ylabel("Y-axis")
ax1.legend()
ax2.plot(x, y1, label="x^2", color='b')
ax2.plot(x, y2, label="exp(x)", color='r')
ax2.set_title("Inverted Plot")
ax2.set_xlabel("X-axis")
ax2.set_ylabel("Y-axis")
ax2.invert_xaxis()
ax2.invert_yaxis()
ax2.legend()
fig.tight_layout()
plt.show()
```



In [ ]: